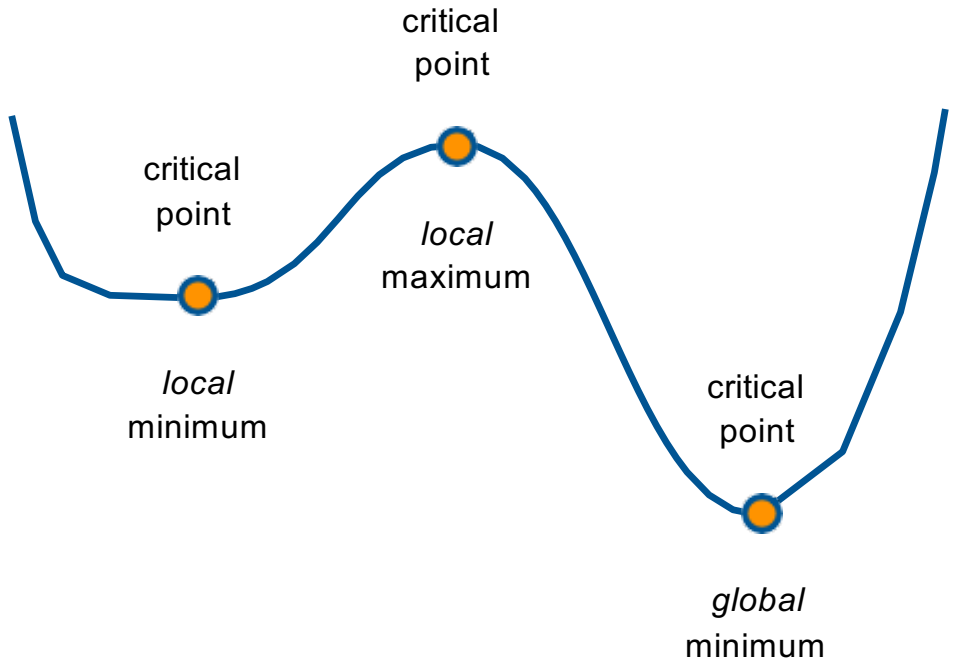
Machine Learning

Lecture 5 - Linear Models II

1 Convexity

When are Critical Points Optimal?

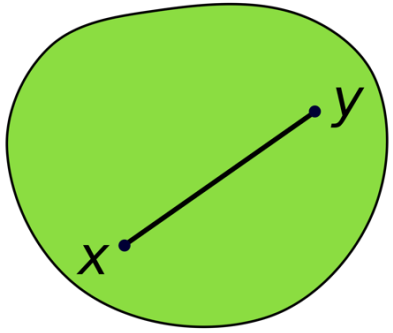
- Gradient descent finds a critical point, but is it a global optimum? In
- general, a critical point may be a local optimum only.
- If a function is convex, then every critical point is a global optimum.

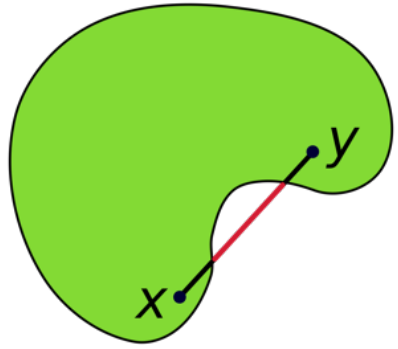


Convex Sets

A set S is convex if
any line segment connecting two points in S lies entirely within S.

$$\mathbf{x}_1, \mathbf{x}_2 \in S$$
$$\Rightarrow \quad \lambda\mathbf{x}_1 + (1-\lambda)\mathbf{x}_2 \in S \qquad \text{for } 0 \leq \lambda \leq 1.$$





Weighted averages or convex combinations of points in S lie within S.

$$\mathbf{x}_1, \ldots, \mathbf{x}_N \in S$$
$$\Rightarrow \lambda_1\mathbf{x}_1 + \cdots + \lambda_N\mathbf{x}_N \in S \qquad \text{for } \lambda_i > 0, \ \lambda_1 + \cdots \lambda_N = 1.$$

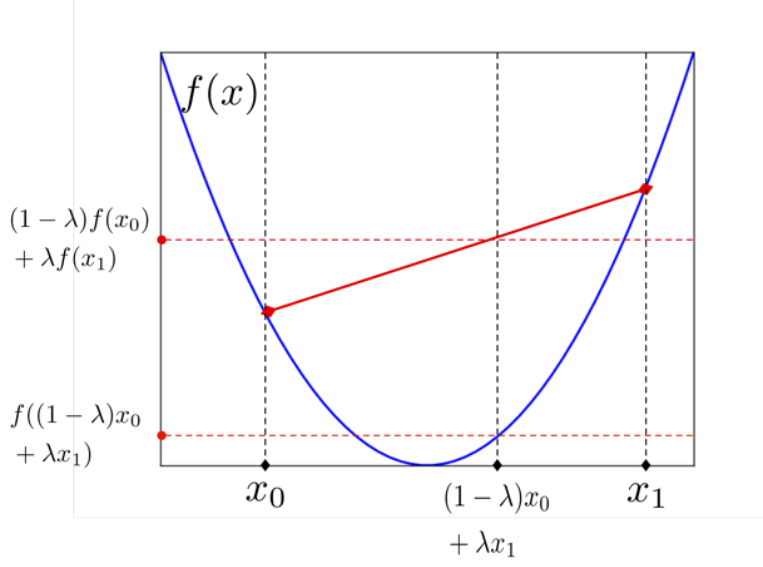
Convex Functions

A function f is convex if

- the line segment between any two points on f's graph lies above f's graph between the two points.
- the set of points lying above the graph of f is convex. for
- any $\mathbf{x}_0$, $\mathbf{x}_1$ in the domain of f,

$$f((1-\lambda)\mathbf{x}_0 + \lambda\mathbf{x}_1) \leq (1-\lambda)f(\mathbf{x}_0) + \lambda f(\mathbf{x}_1)$$

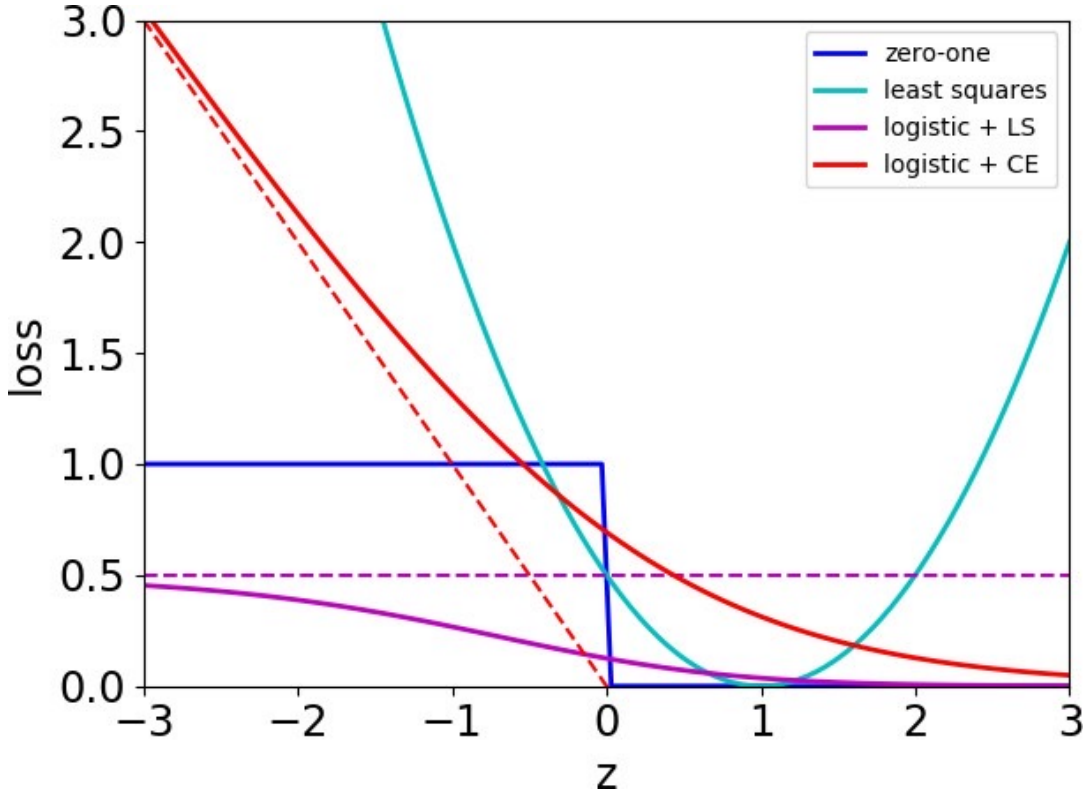
- f is bowl-shaped.



Convex Loss Functions

For linear models, $z = \mathbf{w}^T\mathbf{x} + b$ is a linear function of $\mathbf{w}$ and b. If the loss function is a convex function of z, then it is also a convex function of $\mathbf{w}$ and b.

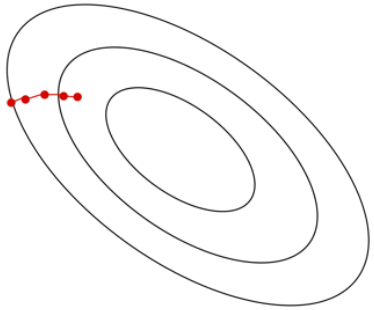
Which loss functions are convex?



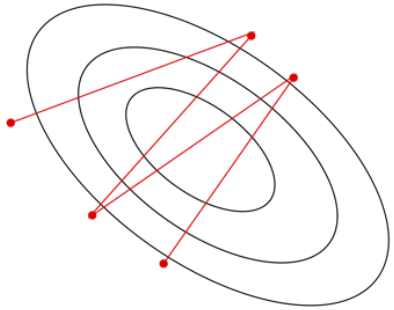
2 Setting the Learning Rate

Impact of Learning Rate on Gradient Descent

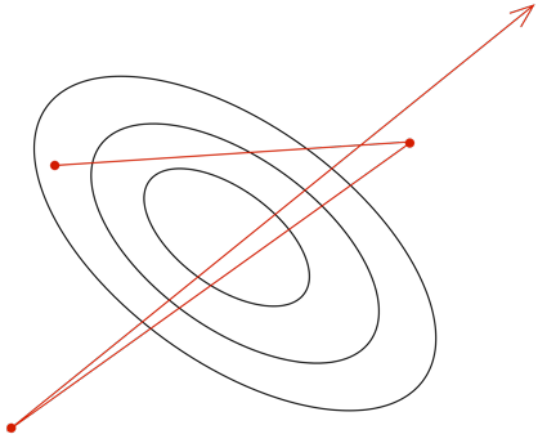
What could go wrong when setting the learning rate?



α too small:
slow progress



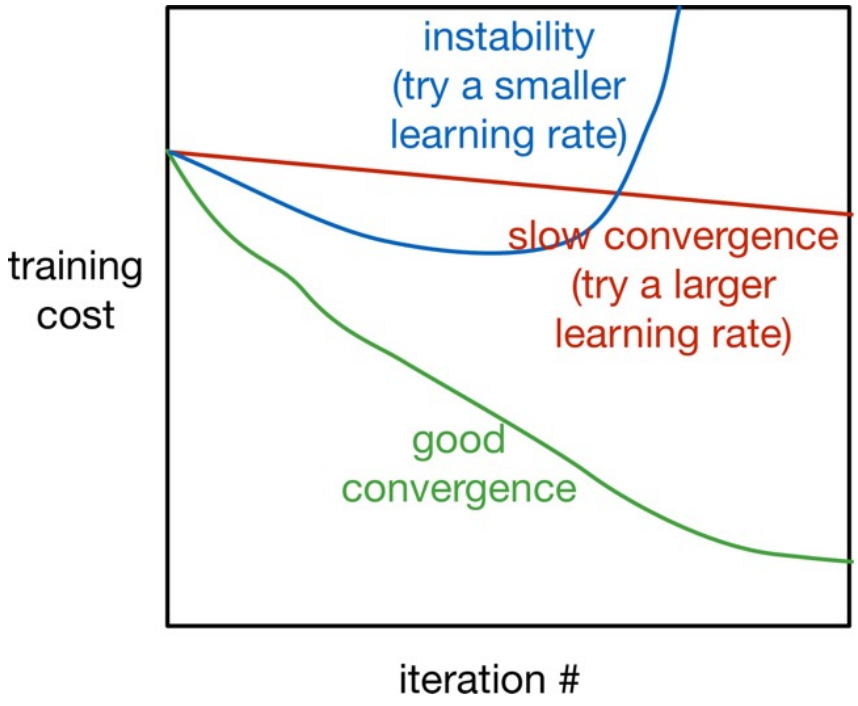
α too large:
oscillations



α much too large:
instability

Finding a Good Learning Rate

- Good values are typically between 0.001 and 0.1.
- Do a grid search for good performance (i.e. try 0.1, 0.03, 0.01, . . .).
- Diagnose optimization problems using a training curve.



Main Takeaways on Learning Rate

How can the learning rate affect the behaviour of gradient descent?

How can we determine the best value for the learning rate?

Main Takeaways on Learning Rate

How can the learning rate affect the behaviour of gradient descent?

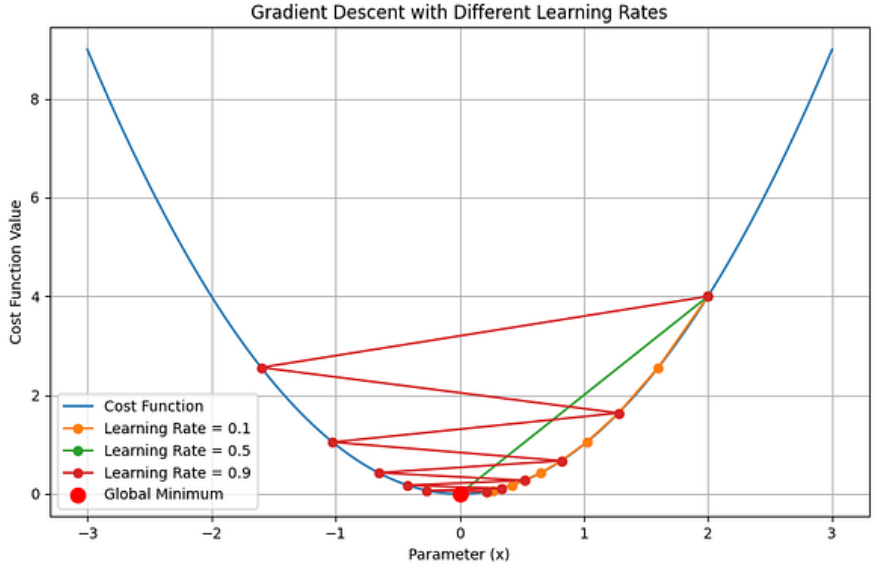
- Too small, slow convergence.
- Too large, oscillations, misses optimum, instability.

How can we determine the best value for the learning rate?

- Grid search
- Training curves can help diagnose optimization problems. Can consider changing the algorithm as well (e.g. line search)
-

Learning rate

the learning rate is usually denoted as α. *It* determines the step size at each iteration while moving towards a minimum of the cost function *J*. If the learning rate is too small, the convergence process will be very slow, potentially requiring a large number of iterations. On the other hand, if the learning rate is too large, it can cause the algorithm to overshoot the minimum, and the cost function might oscillate or even diverge.



Binary Linear Classification

Introducing Binary Linear Classification

$$z = \mathbf{w}^T\mathbf{x} + b$$

$$y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases}$$

- **classification:** predict a discrete-valued target given a D-dimensional input $\mathbf{x} \in \mathrm{R}^D$
- **binary:** predict a binary target $t \in \{0, 1\}$
  - $t = 1$ is a positive example
  - $t = 0$ is a negative example.
  - $t \in \{0, 1\}$ or $t \in \{-1, +1\}$ is for computational convenience.
- **linear:** prediction y is a linear function of $\mathbf{x}$, followed by a threshold r:

Simplified Model

$$z = \mathbf{w}^\top \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Eliminating the threshold r

- Assume that the threshold $r = 0$:

$$\mathbf{w}^\top \mathbf{x} + b \geq r \quad \Longleftrightarrow \quad \mathbf{w}^\top \mathbf{x} + \underbrace{b - r}_{\triangleq b'} \geq 0.$$

Eliminating the bias b

- Add a dummy feature $x_0 = 1$. The weight $w_0 = b$ is equivalent to a bias (same as linear regression)

Modeling Simple Logical Functions

- Examples: NOT, AND.
- Goal is to minimize the training set error. Forget
- about generalizing to a test set for now.

Modeling NOT

NOT

x_0	x_1	t
1	0	1
1	1	0

$$z = w_0 x_0 + w_1 x_1$$

$$y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- Derive two sets of values for w_0, w_1 to classify NOT.
- Which conditions on w_0, w_1 guarantee perfect classification?

Modeling AND

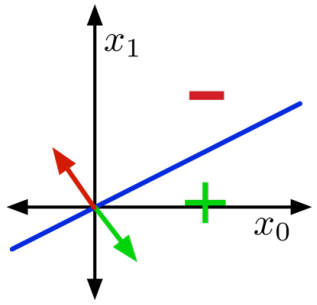
AND

x_0	x_1	x_2	t
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

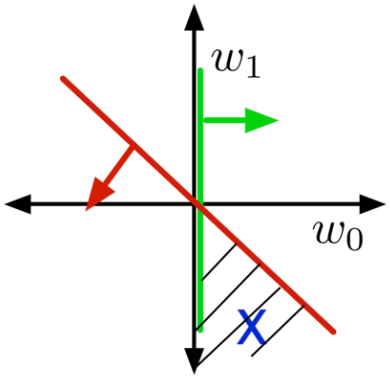
Visualizing NOT in Data Space



x_0	x_1	t
1	0	1
1	1	0

- Each training example is a point in data space.
- The line $\{\mathbf{x}: \mathbf{w}^T\mathbf{x} = 0\}$ defines the decision boundary.
  - A line in 2-D, or a hyper-plane in high dimensions.
  - The boundary passes through the origin (why?)
- Data is linearly separable if a linear decision rule can perfectly separate the training examples.

Visualizing NOT in Weight Space



$w_0 \geq 0$

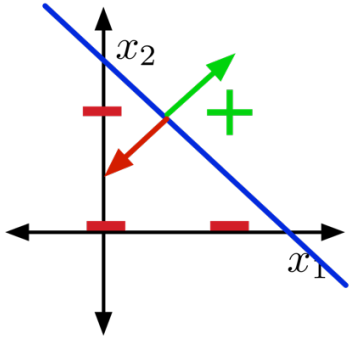
$w_0 + w_1 < 0$

- Each point is a set of values for the weights **w**.
- Each training example **x** specifies a half-space that **w** must lie in to guarantee correct classification.
- The feasible region satisfies all the constraints.
  The problem is feasible if the feasible region is nonempty.

Visualizing AND in Data and Weight Spaces

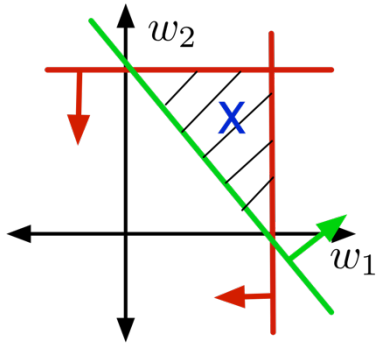
Let's look at a 2-D slice of the 3-D data and weight spaces for AND.

Data Space



- Slice for $x_0 = 1$
- Example Solution:
  $w_0 = -1.5, w_1 = 1, w_2 = 1$
- Decision Boundary:
  $w_0 x_0 + w_1 x_1 + w_2 x_2 = 0$
  $\Rightarrow \quad -1.5 + x_1 + x_2 = 0$

Weight Space



- Slice for $w_0 = -1.5$
- The constraints:
  $w_0 < 0$
  $w_0 + w_2 < 0$
  $w_0 + w_1 < 0$
  $w_0 + w_1 + w_2 \geq 0$

Main Takeaways on Binary Linear Classification

What is binary linear classification (BLC)?

Describe the simplified model for BLC.

How can we model simple logical functions using BLC?

When visualizing BLC in data or weight space,

what do the points, the lines, and the half-spaces represent?

How can we learn the weights for a linearly separable data-set?

Main Takeaways on Binary Linear Classification 1/2

What is binary linear classification (BLC)?

- Predict a binary target by using a linear function of inputs followed by thresholding.

Describe the simplified model for BLC.

$$z = w^T x, y = 1 \text{ if } z \geq 0, y = 0 \text{ otherwise.}$$

How can we model simple logical functions using BLC?

- Derive weights to represent NOT, AND.

Main Takeaways on Binary Linear Classification 2/2

When visualizing BLC in data or weight space,
what do the points, the lines, and the half-spaces represent?

- In data space: points are training examples. a line represents a set of weights.
- In weight space: points are weights. a half-space represents a training example constraining the weights.

How can we learn the weights for a linearly separable data-set?

- linear programming.
- perceptron algorithm.

5 Logistic Regression

Data Isn’t Linearly Separable

What if the data-set isn’t linearly separable?

- Define a loss function.
- Find weights that minimize the average loss over the training examples.

Logistic regression

Logistic regression is used for binary classification where we use sigmoid function, that takes input as independent variables and produces a probability value between 0 and 1.

For example, we have two classes Class 0 and Class 1 if the value of the logistic function for an input is greater than 0.5 (threshold value) then it belongs to Class 1 otherwise it belongs to Class 0. It's referred to as regression because it is the extension of linear regression but is mainly used for classification problems.

In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).

Logistic regression

Despite its name, it’s used for classification rather than regression.
It estimates the probability that an instance belongs to a particular class.
If the estimated probability is greater than 50%, the model predicts that the instance belongs to that class (or vice versa).

Logistic Function – Sigmoid Function

•The sigmoid function is a mathematical function used to map the predicted values to probabilities.
•It maps any real value into another value within a range of 0 and 1. The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the “S” form.
•The S-form curve is called the Sigmoid function or the logistic function.
•In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Types of Logistic Regression

Logistic Regression can be classified into three types:

1.Binomial: In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

2.Multinomial: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as “cat”, “dogs”, or “sheep”

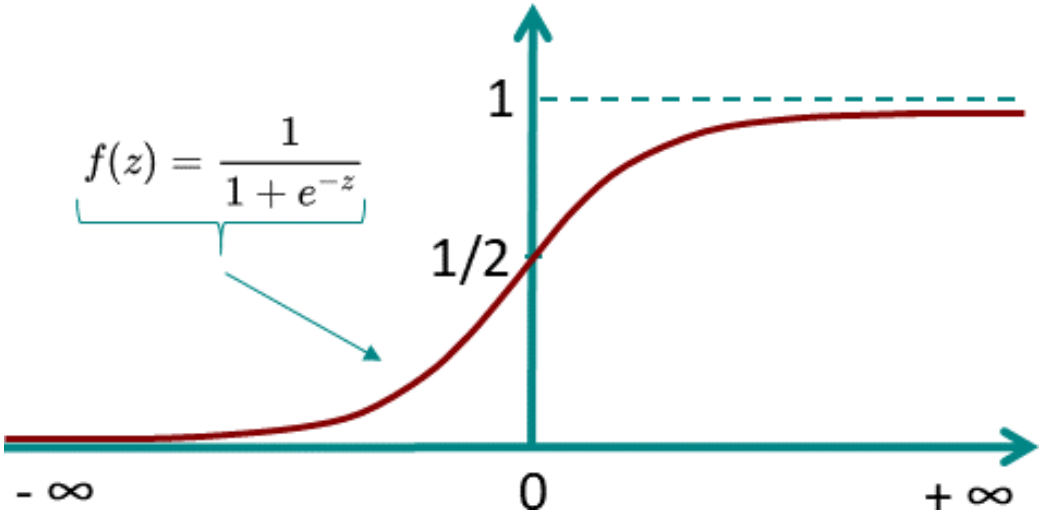
3.Ordinal: In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as “low”, “Medium”, or “High”.

How does Logistic Regression work?

The logistic regression model transforms the linear regression function continuous value output into categorical value output using a sigmoid function, which maps any real-valued set of independent variables input into a value between 0 and 1. This function is known as the logistic function.

$z = w \cdot X + b$

$$y = 1/(1 + e^{-z})$$



Examples

Business example:

For an online retailer, you need to predict which product a particular customer is most likely to buy. For this, you receive a data set with past visitors and their purchases from the online retailer.

Medical example:

You want to investigate whether a person is susceptible to a certain disease or not. For this purpose, you receive a data set with diseased and non-diseased persons as well as other medical parameters.

Columns: ‘Outlook’, ‘Temperature’, ‘Humidity’, ‘Wind’ and ‘Play’ (target feature).
The categorical columns (Outlook & Windy) are encoded using one-hot encoding while the numerical columns are scaled using standard scaling (z-normalization).

Training Set

0	1	0	0	1.80	0.50	0	No
1	1	0	0	1.02	1.02	1	No
2	0	1	0	1.49	-0.24	0	Yes
3	0	0	1	-0.56	1.66	0	Yes
4	0	0	1	-0.88	-0.03	0	Yes
5	0	0	1	-1.35	-1.08	1	No
6	0	1	0	-1.51	-1.61	1	Yes
7	1	0	0	-0.25	1.55	0	No
8	1	0	0	-0.72	-1.08	0	Yes
9	0	0	1	0.23	-0.03	0	Yes
10	1	0	0	0.23	-1.08	1	Yes
11	0	1	0	-0.25	1.02	1	Yes
12	0	1	0	1.17	-0.56	0	Yes
13	0	0	1	-0.41	-0.03	1	No

Test Set

14	1	0	0	1.17	0.81	1	No
15	0	0	0	0.07	1.24	0	Yes
16	0	1	1	0.38	0.50	0	Yes
17	1	0	0	0.70	-0.56	1	No
18	1	0	0	1.33	1.24	0	No
19	0	1	1	-1.04	1.02	1	No
20	0	0	0	1.80	0.50	1	Yes
21	0	1	1	-0.09	0.81	0	Yes
22	1	0	0	2.28	-1.61	1	Yes
23	0	0	0	0.54	-1.08	0	Yes
24	1	0	0	0.86	-2.14	0	Yes
25	0	0	0	1.02	1.55	1	Yes
26	0	1	1	-1.20	-1.08	0	No
27	0	0	0	1.65	-0.24	0	Yes

Example

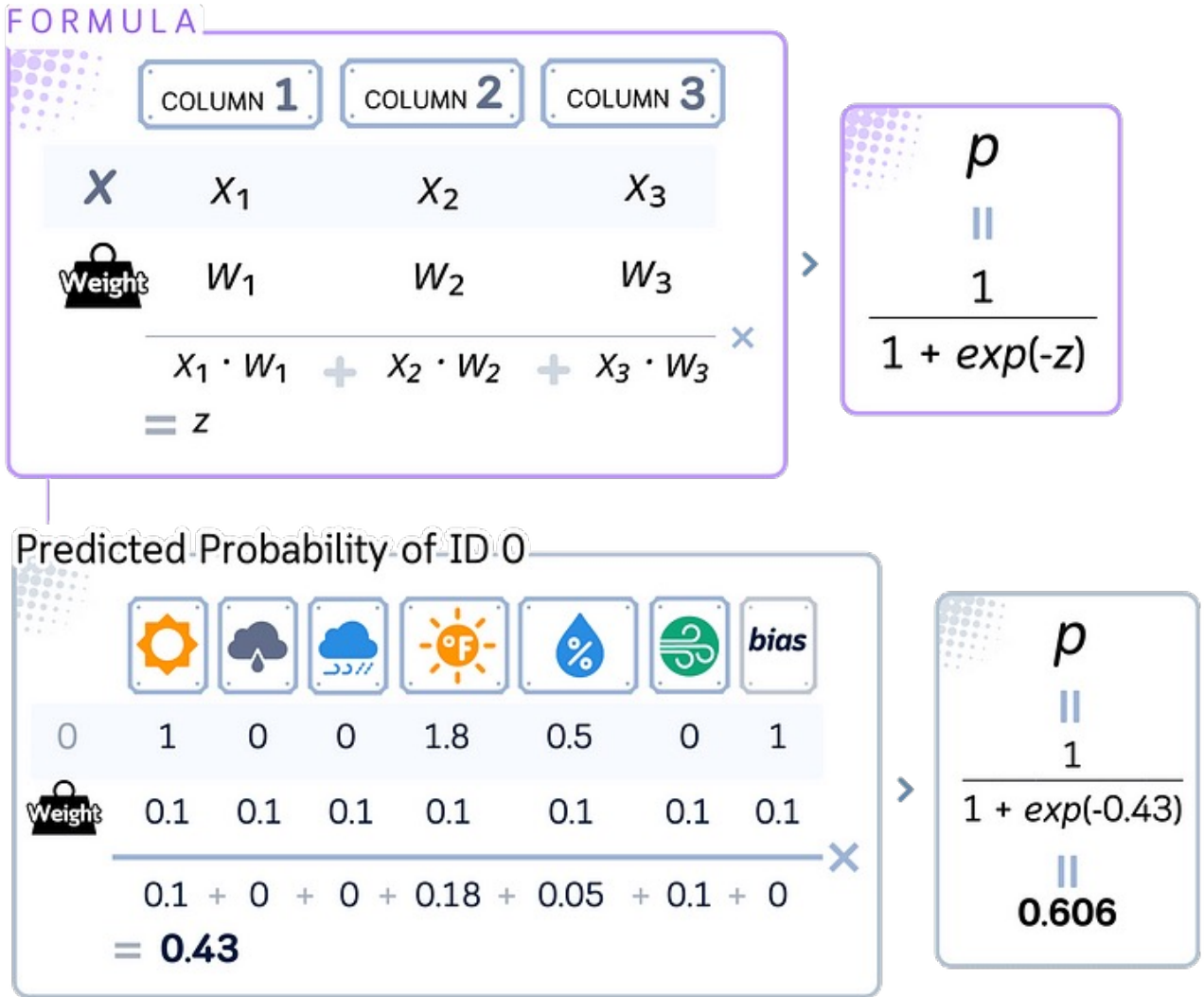
1.Initialize the weights (often to small random values).



Example

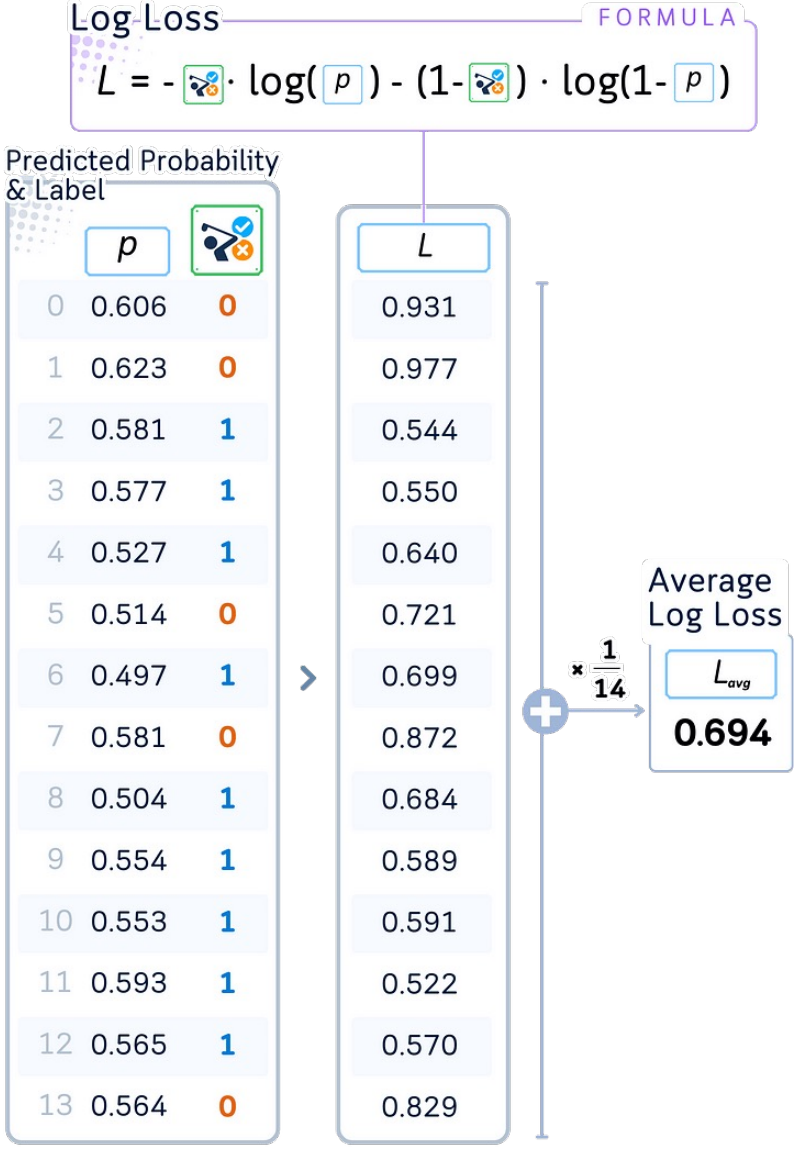
2. For each training example:

a. Calculate the predicted probability using the current weights.



Example

b. Compare this probability to the actual class label by calculating its log loss.



Example

3. Update the weights to minimize the loss (usually using some optimization algorithm, like gradient descent. This include repeatedly do Step 2 until log loss cannot get smaller).

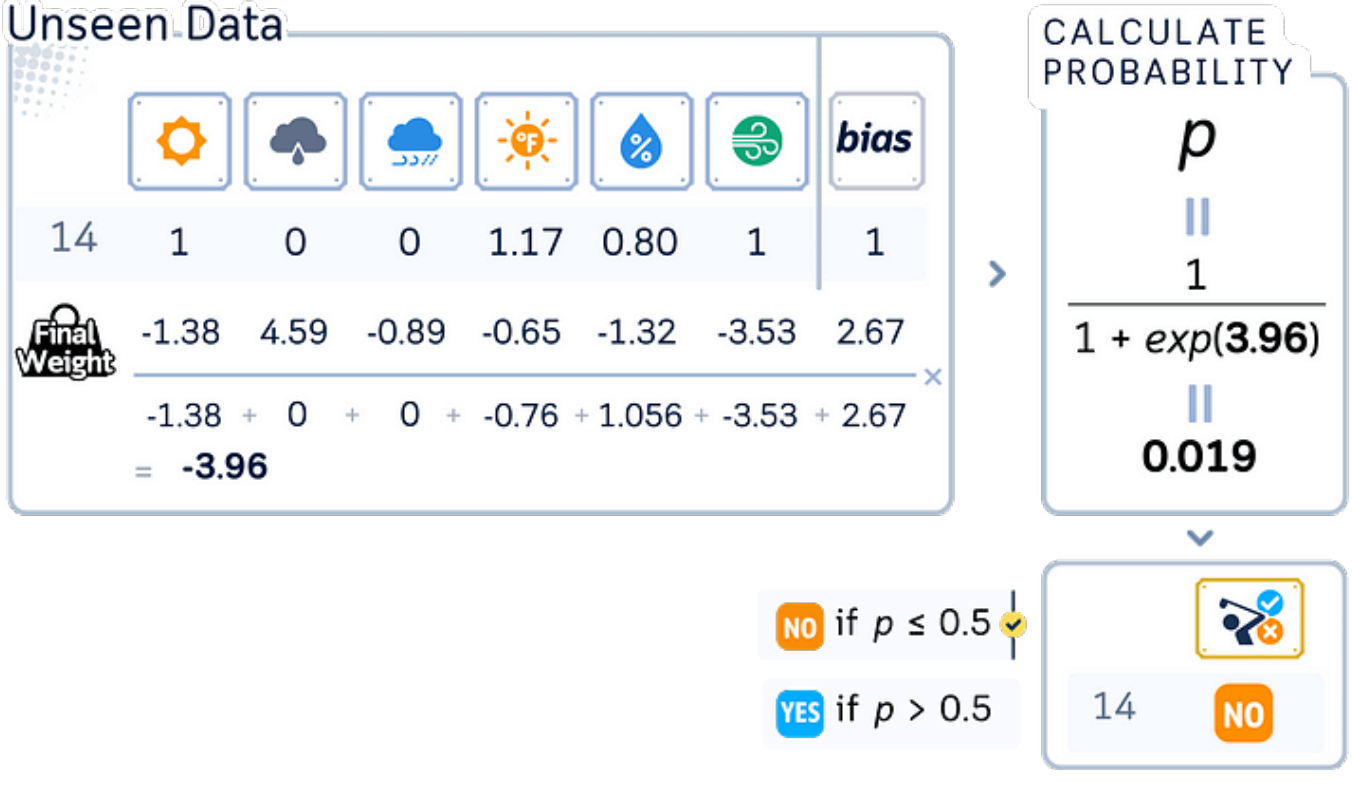


Example

Classification Steps

Once the model is trained:

1. For a new instance, calculate the probability with the final weights (also called coefficients), just like during the training step.
2. Interpret the output by seeing the probability: if $p \geq 0.5$, predict class 1; otherwise, predict class 0



Thanks